Análise de operações em estruturas de dados

Prof. Pablo De Chiaro

algoritmos e estrutura de dados 2 (AEDII)

_aula 4

Objetivos da Aula

Finalizar a atividade e conceitos da aula anterior sobre complexidade.

Fechar o bloco introdutório de complexidade (Big-O).

Introduzir arrays (vetores) como a principal estrutura linear indexada.

Relacionar operações (acesso, busca, inserção, remoção) com seu custo computacional.

O que já estudamos até aqui?

O (1) e O (n)

Tempo constante para operações diretas e tempo linear para travessias simples em estruturas.

O que já estudamos até aqui?

O (log n) e O (n log n)

Divisão e conquista, busca binária e os melhores algoritmos gerais de ordenação (Merge Sort).

O que já estudamos até aqui?

O (n^2) e O (2^n)

Custo quadrático (laços aninhados) e explosão combinatória/crescimento exponencial indesejado.

Todo algoritmo depende só da lógica?

Não. A forma como os dados são organizados também influencia dramaticamente o seu desempenho.

Como identificar a operação dominante?

Focar no trecho de código que mais cresce com a entrada.

Quando algo é linear?

Quando precisamos percorrer cada item da entrada pelo menos uma vez, custando O (n).

Por que nem todo laço leva a O (n^2) ?

É necessário analisar a dependência entre os laços e se eles percorrem N iterativamente um dentro do outro.

O que torna O (2^n) tão custoso?

O teste de todas as combinações possíveis causa a "explosão combinatória".

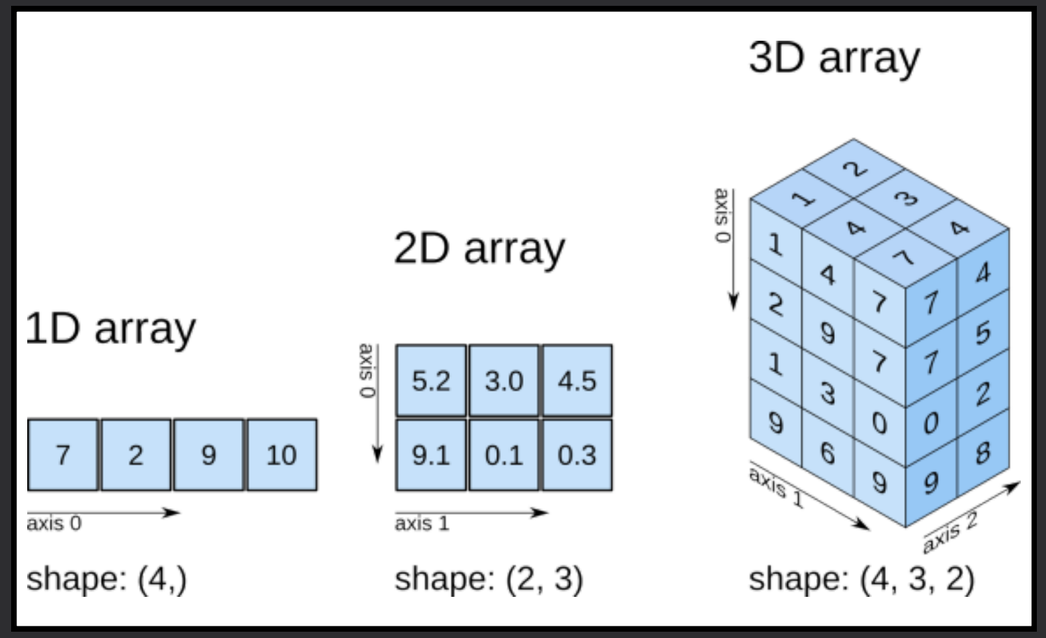
Big-O resumido

Situação Analisada	Complexidade de Tempo
Acessar posição específica de imediato	$O(1)$
Percorrer todos os elementos	$O(n)$
Dividir o problema ao meio a cada passo	$O(\log n)$
Ordenar eficientemente (Merge Sort)	$O(n \log n)$
Comparar tudo com tudo (Bubble Sort)	$O(n^2)$
Testar todas as combinações possíveis	$O(2^n)$

Agora entra a estrutura de dados

A mesma operação pode ser extremamente rápida ou incrivelmente lenta **dependendo da estrutura de dados usada**.

Entender a complexidade de tempo não é apenas sobre a lógica da sua função, mas sobre as características mecânicas da estrutura escolhida para armazenar a informação.



O que é um Array? (Vetor)

Estrutura Linear: Elementos são armazenados de **forma contígua e sequencial na memória**.

Posições Indexadas: Cada elemento recebe um índice numérico. Normalmente, o índice inicial é zero.

Acesso Direto: Permite acessar qualquer posição de forma imediata através do seu índice.

JS

```js
let notas = [7.5, 8.0, 6.5, 9.0];

console.log(notas[0]); // Saída: 7.5
console.log(notas[3]); // Saída: 9.0
```

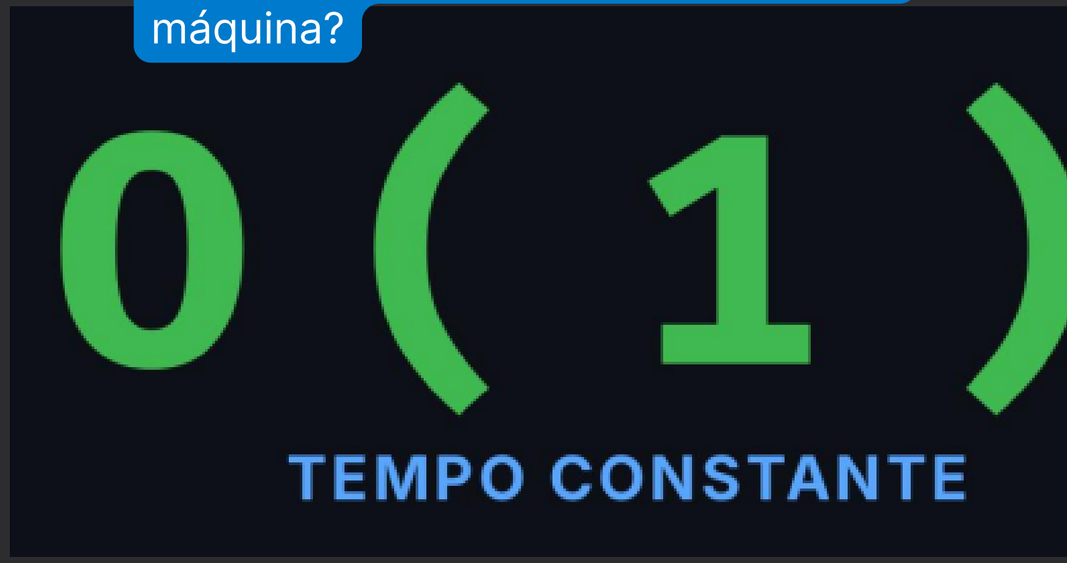
Por que o ato de executar notas[3] possui alta performance na máquina?

Acesso por Índice

Acessar um elemento por índice é uma operação direta. O computador sabe o endereço inicial do array e simplesmente calcula a posição alvo matematicamente.

Conclusão: Não é preciso percorrer todos os elementos antes dele. Acesso por índice em um array tende a ser constante.

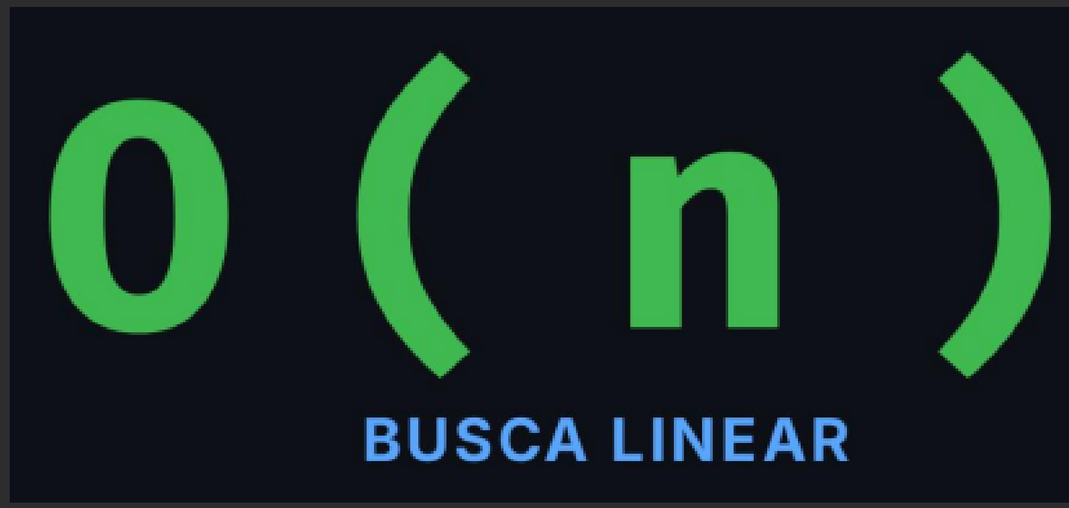
Por que o ato de executar notas[3] possui alta performance na máquina?



Busca por Valor

Percorrendo a Estrutura Para descobrir se um determinado valor está presente no array, geralmente precisamos verificá-lo elemento por elemento, do início ao fim.

Conclusão: No pior caso (o item está no final ou não existe), somos forçados a passar por todos os elementos. O custo é diretamente proporcional ao tamanho do array.



Exemplo de Busca Linear

O laço de repetição itera sobre o array incrementando a variável i.

A cada iteração, é feito um acesso **O(1)** na posição (**num[i]**), seguido de uma comparação lógica.

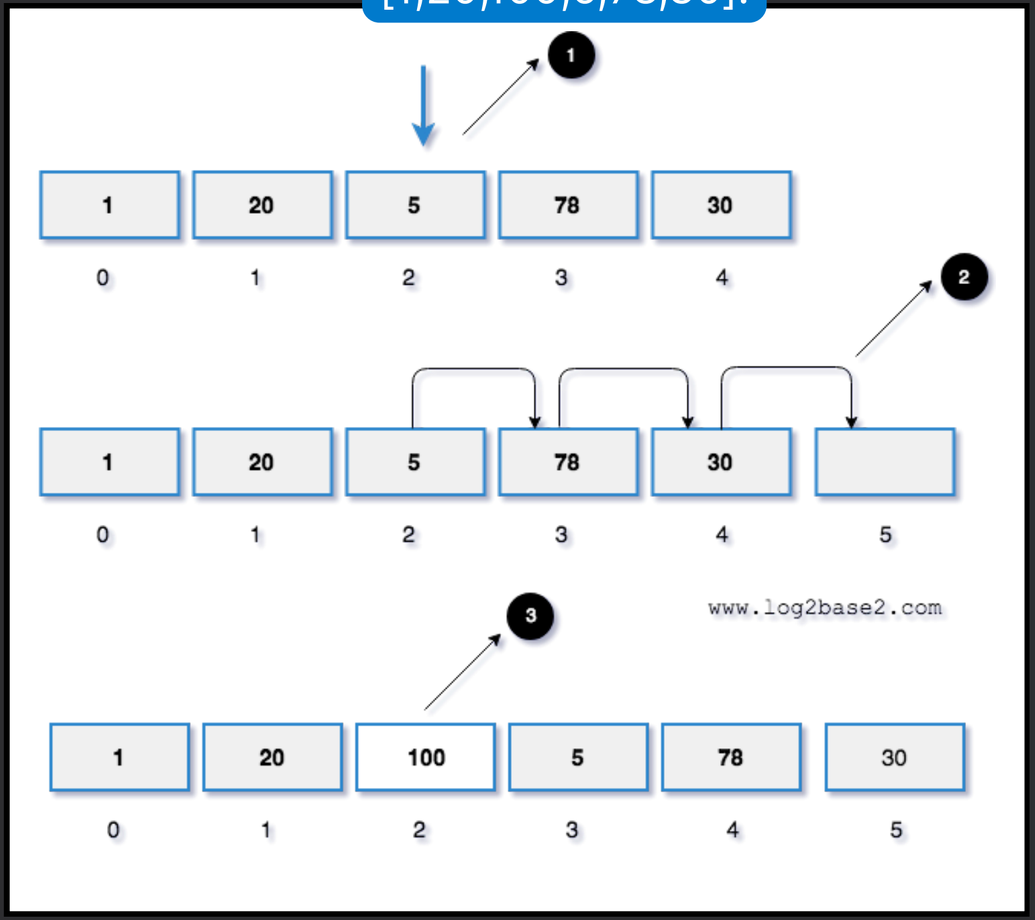
Como iteramos potencialmente N vezes, o bloco todo escala linearmente de acordo com a regra dominante.

```js
let num = [10, 20, 30, 40, 50];
let procurado = 40;
let achou = false;

for (let i = 0; i < num.length; i++) {
    if (num[i] === procurado) {
        achou = true;
        break;
    }
}
```

Inserção

Exemplo: [1,20,5,78,30] – Inserir 100 no inicio/meio Move todos para a direita: [1,20,100,5,78,30].



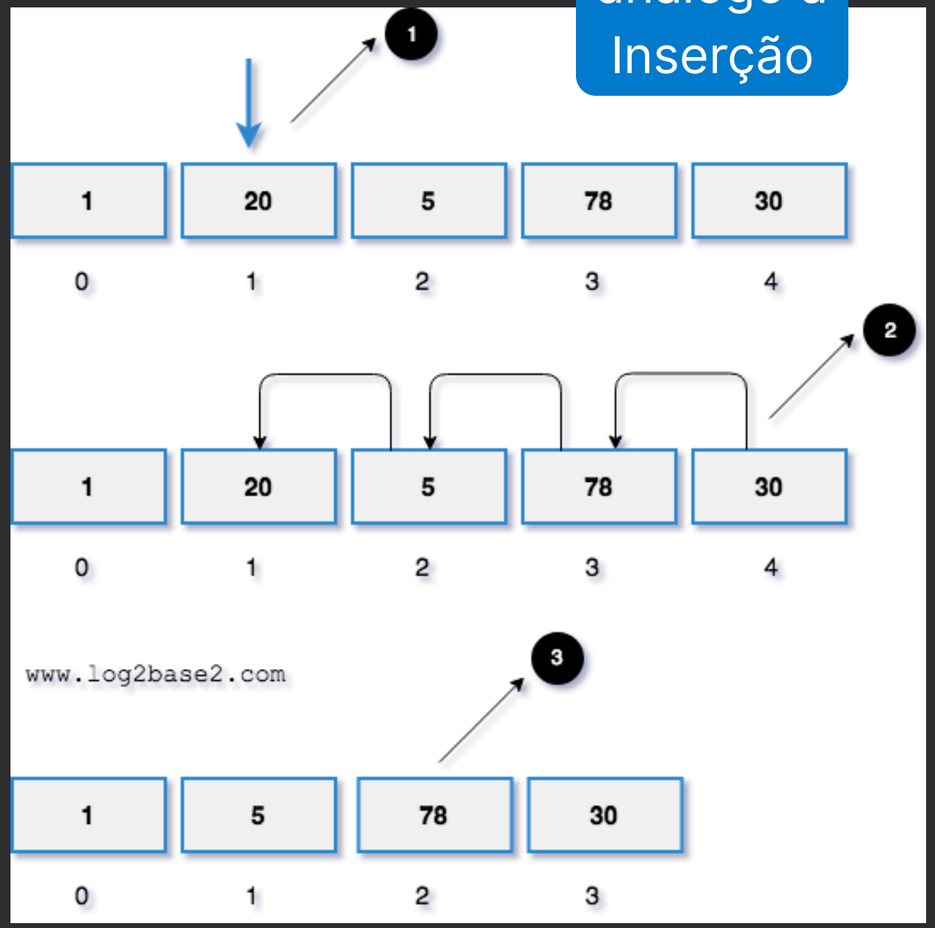
No Final: Inserir um elemento ao fim do array tende a ser simples e custa apenas 0 (1) namaioria das vezes.

No Início ou Meio: Exige um "deslocamento" (shift) dos elementos existentes para abrir espaço.

Conclusão: A inserção nestas posições indesejadas pode custar até 0 (n).

Remoção

Exemplo análogo à Inserção



No Final: Remover o último elemento do array é direto, sem dor de cabeça, resultando em 0(1).

No Início ou Meio: Exige a reorganização da estrutura. Todos os elementos à direita do removido precisam "voltar" um passo para preencher o vazio.

Conclusão: O custo de remoção no início/meio tende a 0(n).

Síntese: Operações em Arrays

Operação em Array	Complexidade Típica (Pior Caso)
Acesso por índice (ex: array[5])	O(1)
Busca por valor (linear)	O(n)
Inserção no final	O(1)
Inserção no início/meio	O(n)
Remoção no início/meio	O(n)
Percorrer todo o array	O(n)

Uma estrutura de dados não é uma solução mágica perfeita para tudo.

Arrays são excepcionalmente bons para acesso direto, mas podem ser altamente ineficientes para inserções e remoções sistêmicas dependendo da posição em que a operação ocorre.

Exercícios

Q1. Crie um array com 5 nomes e exiba o elemento da posição 2. Identifique a complexidade.

Q2. Crie um array com 100 valores aleatórios faça uma busca para verificar se o valor 25 está presente. Identifique a complexidade do pior caso.

Q3. Explique por que acessar por índice é mais rápido do que procurar um valor em array não ordenado.

Q4. Explique o que acontece fisicamente na estrutura quando um novo item é inserido no início.

Q5. Programe um script completo em JS (armazenar 5 números, mostrar primeiro, último, somar tudo, e buscar o valor 10).

Próxima aula

Arrays versus Listas Ligadas (Linked Lists)

Quando uma estrutura é mais adequada que a outra?
Quais são os prós e contras em relação ao custo das operações?

Prof. Pablo De Chiaro
algoritmos e estrutura de dados 2 (AEDII)